

Code review — variante-bac

Runde de review pe proiectul tău, cea mai nouă sus.

Un singur fișier, runda cea mai nouă sus. Rundele vechi rămân dedesubt.

Runda 2 — 2026-09-23 — sha 02640ce

Patru commit-uri: `a9c3abc var24 corectat`, `d068314 tema`, `7bbed5c finish tema`, `02640ce finish var8`.

16 variante noi (18, 19, 23, 25, 26, 40, 41, 42, 43, 46, 51, 52, 55, 62, 64, 66) și **5 atinse** (3, 5, 8, 24, 39). Am verificat tot, inclusiv ce a rămas deschis din runda 1.

Codul NU a fost modificat — totul e doar aici.

Verificat prin **compilare și rulare**, nu prin citire: `g++ -std=c++17 -Wall -Wextra`, fiecare header compilat separat, apoi rulat cu datele din enunțuri **și** cu date alese special ca să lovească bug-urile.

Verdictul pe scurt.

Poarta de compilare: trecută. Toate cele 30 de headere compilează. A doua rundă la rând fără „nu compilează”. Au ieșit warning-uri în 4 fișiere, iar 3 dintre ele sunt bug-uri reale.

Algoritmii s-au făcut buni. Din cele 16 variante noi, **13 dau răspunsul corect** pe datele din enunț. Asta e o săritură reală față de runda 1, unde din 4 fișiere ale temei unul singur mergea. Euclid în `varianta66`, inversarea prin apeluri repetate de `shift` în `varianta23`, concatenarea cu `ordin()` în `varianta55` — sunt rezolvări bune, nu noroc.

Dar s-a mutat problema. Runda 1: „te oprești la un pas de final”. Runda 2: **rezolvi altă problemă decât cea cerută.**

ce cere enunțul	ce ai scris
„folosind apeluri utile ale subprogramului <code>cif</code> ”	<code>cif</code> definit, apelat de 0 ori — treaba o face <code>frecventaCif</code>
„subprogramul <code>sub</code> va afișa pe ecran...”	<code>sub</code> gol; treaba o face <code>interclasare</code>
„citește din fișierul <code>bac.in</code> / <code>date.in</code> / <code>NUMERE.IN</code> ”	date scrise în cod; zero <code>fstream</code> în tot repo-ul
„valoarea afișată va avea cel mult 3 cifre după virgulă”	<code>4, 8 ,5.66667</code>
„se va afișa mesajul <code>NU EXISTA</code> ”	<code>nu exista numere prime</code>

La bac, subiectul III punctul b) nu punctează „răspunsul corect”. Punctează „răspunsul corect, obținut cu subprogramul de la punctul a), pe datele din sursa indicată”. Un algoritm perfect care ocolește subprogramul cerut ia punctele de la a) și zero de la b).

Vechiul tipar n-a dispărut, s-a concentrat. Trei `if` -uri cu corpul gol, în trei fișiere: `varianta3.h:54`, `varianta25.h:22`, `varianta8.h:20`. Exact aceeași mișcare ca în runda 1 — cadrul scris, decizia din mijloc lipsă.

Starea constatărilor din runda 1

#	fișier	stare
B1 parametri prin copie	<code>varianta24.h</code>	✅ ÎNCHIS — <code>int &mini,</code> <code>&maxi, &sum</code>
B2 <code>for (int i = i; ...)</code>	<code>varianta24.h</code>	✅ ÎNCHIS — <code>i = 1</code>
B3 nu afișează / împărțire întreagă	<code>varianta24.h</code>	🟡 PE JUMĂTATE — vezi B3 mai jos
B4 cifra 9 nenumărată	<code>varianta3.h</code>	⬜ dispărut odată cu codul rescris
B5 palindromul	<code>varianta3.h</code>	🔴 ÎNRĂUTĂȚIT — vezi B1 mai jos
B6 <code>f</code> nu e apelat	<code>varianta5.h</code>	🟡 PE JUMĂTATE — vezi B4 mai jos
B7 <code>int</code> fără <code>return</code>	<code>varianta39.h</code>	✅ ÎNCHIS — <code>void</code> <code>solutie3()</code>
B8 <code>sub</code> schelet	<code>varianta8.h</code>	🔴 NEATINS — vezi B2 mai jos
M1 coliziuni de nume	tot repo-ul	🔴 AGRAVAT — vezi M2
M2 <code>sort</code> peste <code>std::sort</code>	<code>varianta1/12</code>	🔴 neatins
M3 prompturi la tastatură	5 locuri	🔴 neatins
M4 citire din <code>bac.in</code>	<code>varianta39.h</code>	🔴 neatins — vezi M1
M5 <code>nule</code> afișează	<code>varianta20.h</code>	🔴 neatins
C1–C5	—	🔴 toate neatinse

Trei 🔴 închise curat, două pe jumătate. Cele închise sunt exact cele unde ți-am dat linia de cod. Cele pe jumătate sunt acolo unde constatarea avea două lucruri în ea și l-ai făcut pe primul.

Ce am rulat, ca să vezi dovada

```
V3 : (nimic)                                trebuia 83155743 sau 0
V5 : 7 / 1125 2 314 101 37 225 12  -> 2 37 101                corect
V5 : 3 / 4 6 8      -> "nu exista numere prime" trebuia NU EXISTA
V8 : a={2,4,8,10,14} b={3,5,11}      -> 2 3 4 5 8 11 14        corect
V8 : a={2,4}        b={1}            -> 1                      trebuia 1 2
V8 : a={2,4,6}      b={1}            -> 1                      trebuia 1 2
V18 : {12,7.5,6.5,3,8.5,7.5}         -> 4                      corect
V19 : {12,-7.5,6.5,-3,-8,7.5}  -> -7.5 -3 -8 12 6.5 7.5        corect
V23 : {1,2,3,4,5}                -> 5 4 3 2 1                corect
V24 : n=5, {1,9,4,8,5}      -> "4, 8 ,5.66667" trebuia 5.667
V26 : exemplul din enunț  -> "10 2 19 9 15 0" trebuia fara 0
V39 : 5 / 4 3 6 1 8  -> DA · 5 / 8 3 6 1 4  -> NU                corect
V40 : {10,1133,12,331,42,1354,221,13} -> DA                      corect
V41 : {23,44,76,26,65}        -> 1                            corect
V42 : {5,2,5,4,1,3} k=4      -> 17                            corect
V43 : {3,12,7,1,4,3}         -> 4                            corect
V46 : {100,9,400,56,7,10} k=5 -> 100 400 56                corect
V51 : {16,22,3,6,1}  -> 4 perechi, cate una pe linie          corect
V52 : {16,17,8,31,8,2,10}    -> 4                            corect
V52 : {1,2,3} (toate distincte) -> 2                      trebuia 3
V55 : 3 / 12 5 100 / 7 34 9   -> 712 534 9100                corect
V55 : 1 / 5 / 0              -> 5                      trebuia 50
V62 : n=8                    -> 1 1 3 5 13 21 55 89            corect
V64 : {4,8,3,9,4}            -> 4                            corect
V66 : {12,36,48,6,60}        -> 6                            corect
V25 : f(a,4,k)  -> returneaza 1 (gunoi), k ramane neschimbat
```

Critice

B1 — `varianta3.h` : nu mai afișează nimic, iar `cif` nu e apelat niciodată

`varianta3.h:54` (corp gol) · `varianta3.h:63` (`cif` nefolosit) · `varianta3.h:80–83` (`solutie4` goală)

CUM E ACUM: rulat, programul afișează ecran gol. În runda 1 dădea un răspuns greșit; acum nu dă niciunul. Warning-urile spun exact unde s-a oprit scrisul: „*unused variable 'nou'*”, „*unused variable 'n'*”, „*unused variable 'f'*”.

```
if (verificareaFormariiPalindrom(f)) {  
    int nou = 0;  
    //0 1 2 3 4 5 6 7 8 9  
    //2 0 2 0 0 2 0 0 2 0  
    //85200258  
}  
// <- corpul se termina aici
```

DE CE: sunt două lucruri diferite aici și al doilea e mai grav.

Primul: ai **rescris** în loc să reparați. B4 și B5 din runda 1 erau în algoritmul vechi; algoritmul vechi nu mai există, deci bug-urile au dispărut — dar odată cu ele a dispărut și răspunsul. Când rescreeți, pornești de la zero pe toate cele trei părți (frecvențe, test de imposibilitate, construcția numărului), nu de la două din trei. Rescrierea nu e greșită ca decizie; e greșită ca **moment**, pentru că n-ai dus-o la capăt.

Al doilea: enunțul spune „folosind apeluri utile ale subprogramului `cif`”. Tu ai scris `cif` corect — și apoi ai scris `frecventaCif`, care face aceeași treabă pentru toate cele 10 cifre deodată. `cif` nu e apelat nicăieri. La bac asta nu e o chestiune de stil: punctul b) verifică explicit *apelul*, pentru că asta testează el — că știi să descompui problema în subprogram + program principal. `frecventaCif` e un al doilea subprogram, nu un apel al primului.

Apelul util arată așa: frecvența cifrei `c` în `n` este `cif(n, c)`.

```
// ACUM
void frecventaCif(int f[], int nr) {
    while (nr != 0) { int cif = nr % 10; f[cif]++; nr = nr / 10; }
}

// CORECT – bucla de frecvente devine 10 apeluri ale lui cif
for (int c = 0; c <= 9; c++)
    f[c] = cif(n, c);
```

Și mai lipsesc două lucruri pe care le aveai scrise în runda 1 și nu le-ai mutat în varianta nouă: afișarea lui `0` când palindromul nu se poate forma, și construcția efectivă a numărului (jumătatea descrescătoare, apoi cea crescătoare).

O observație pe `verificareaFormariiPalindrom` : cere ca **toate** frecvențele să fie pare. Pentru enunțul ăsta e corect, fiindcă `n` are exact 8 cifre — număr par de cifre, deci nicio cifră nu are voie să rămână la mijloc. Dacă refolosești funcția la un număr cu lungime impară, regula devine „cel mult o cifră cu frecvență impară”.

B2 — `varianta8.h` : `sub` e tot gol, iar `interclasare` pierde ultimul element

`varianta8.h:20–29` (`sub` gol) · `varianta8.h:84–95` (bug-ul)

CUM E ACUM: pe datele din comentariile tale merge:

```
a={2,4,8,10,14}  b={3,5,11}  ->  2 3 4 5 8 11 14      corect
```

Pe date în care un tablou are un singur element, nu:

```
a={2,4}    b={1}    ->  1          trebuia  1 2
a={2,4,6}  b={1}    ->  1          trebuia  1 2
```

DE CE: în aceeași funcție, `i` și `j` înseamnă două lucruri diferite, în funcție de unde te uiți.

În pasul de pornire (liniile 84–91), după ce iei un element **avansezi indicele**: `c[dimC] = b[j]; j++;` . Deci după pasul ăsta, `j` arată *după* elementul consumat.

În bucla principală (liniile 95-118), după ce ieși un element **nu avansezi nimic**: `c[dimC] = b[j];` și atât. Deci acolo `j` arată pe elementul consumat, iar `while`-ul dinăuntru (`b[j] < c[dimC-1]`) e cel care îl sare la pasul următor. Ambele convenții funcționează separat — dar sunt convenții diferite.

Condiția `while (i < dimA && j < dimB)` a fost scrisă pentru a doua convenție: „mai am elemente neconsumate în amândouă”. Cu prima convenție, `j++` pe un tablou cu un singur element duce `j` la `1`, iar `1 < 1` e fals. Bucla nu pornește niciodată. Tabloul `a`, neatins, se pierde întreg.

Și separat: `sub` — subprogramul pe care îl cere de fapt enunțul — e tot ce era în runda 1: trei variabile declarate, niciuna citită, un `cout << endl`. Compilatorul le enumeră pe toate: „*unused variable 'j'*”, „*unused parameter 'a'*”. Ce ai scris în `interclasare` e exact conținutul lui `sub`; trebuie doar mutat acolo.

```
// ACUM – pasul de pornire avanseaza, bucla nu
if (a[i] > b[j]) { c[dimC] = b[j]; j++; }
else           { c[dimC] = a[i]; i++; }
dimC++;
while (i < dimA && j < dimB) { ... }

// CORECT – aceeași convenție peste tot: nu avansezi la luare,
// avansezi doar când cauți următorul candidat
if (a[i] > b[j]) c[dimC] = b[j];
else           c[dimC] = a[i];
dimC++;
while (true) { ... } // ieșirea rămâne pe break-urile din interior
```

B3 — `varianta24.h`: afișează `4, 8 ,5.66667` în loc de `5.667`

`varianta24.h:77` (afișarea) · `varianta24.h:56-57` (datele)

CUM E ACUM: B1 și B2 din runda 1 sunt închise corect și media se calculează corect (`5.66666...`). Dar pe ecran apare:

```
4, 8 ,5.66667
```

DE CE: trei lucruri, toate în afara algoritmului, care e bun.

`mini` și `maxi` de pe linia 77 sunt rămășițe de depanare — după al doilea apel al lui `p` ele nu mai sunt minimul și maximul din enunț (alea au fost șterse), ci minimul și maximul din **ce a rămas**. Nu le

cere nimeni; ies pe ecran și strică tot.

`5.66667` în loc de `5.667` e o chestiune de cum tipărește `cout` numerele reale. Implicit el folosește *cifre semnificative*, șase la număr — le numără de la prima cifră nenulă, nu de la virgulă. Enunțul cere *cifre după virgulă*, cel mult 3. Sunt două lucruri diferite, iar schimbarea se face explicit, cu `fixed` (numără de la virgulă) plus `setprecision(3)` (câte), amândouă din `<iomanip>`.

Al treilea: `n` și `x` sunt scrise în cod (`int n = 5;` , `int x[100] = {1,9,4,8,5};`). Enunțul spune „citește de la tastatură o valoare naturală nenulă n ... apoi cele n elemente”. Rulat cu alt set de date, programul dă tot `5.667`.

```
// ACUM
int n = 5;
int x[100] = {1,9,4,8,5};
...
cout << mini << " , " << maxi << " , " << medie << endl;

// CORECT
#include <iomanip>
int n;
cin >> n;
int x[100];
for (int i = 0; i < n; i++) cin >> x[i];
...
cout << fixed << setprecision(3) << medie;
```

B4 — `varianta5.h`: mesajul e tot `nu exista numere prime`

`varianta5.h:54`

CUM E ACUM: algoritmul e reparat și merge:

```
7 / 1125 2 314 101 37 225 12    ->  2 37 101        corect
3 / 4 6 8    ->  "nu exista numere prime"    trebuia  NU EXISTA
```

DE CE: B6 din runda 1 avea două părți: apelul lipsă (`f(x) == x`) și mesajul. Ai făcut-o pe prima. Pe a doua o aveai scrisă, în același paragraf, cu litere mari.

Ăsta e tiparul pe care ți-l semnalez a treia oară, și e același la runda 8 pe `victor-vectori` : **repari linia care ți-a fost arătată, nu regula din spatele ei**. Diferența dintre cele două se vede abia la runda următoare, când reapare aceeași constatare cu alt număr de linie.

Și de ce contează exact aici: la bac lucrarea ta e comparată automat, caracter cu caracter, cu output-ul așteptat. `nu exista numere prime` și `NU EXISTA` sunt două șiruri diferite. Algoritmul corect, apelul util scris, sortarea corectă — și punctul b) iese 0, pentru 12 caractere.

```
// ACUM
cout << "nu exista numere prime";

// CORECT
cout << "NU EXISTA";
```

B5 — `varianta25.h` : subprogramul `f` e schelet și întoarce gunoi

`varianta25.h:17` (antetul) · `varianta25.h:22–24` (corp gol)

CUM E ACUM: fișierul are un singur lucru în el, subprogramul cerut, și e gol pe dinăuntru. Rulat, returnează `1` și lasă `k` neschimbat:

```
f(a,4,k) -> a returnat 1, k a ramas -99      trebuia k = 42
```

Trei warning-uri, toate în patru linii de cod: „*parameter ‘k’ set but not used*”, „*unused variable ‘p’*”, „*non-void function does not return a value*”.

DE CE: antetul contrazice enunțul în două locuri, iar asta e util de văzut înainte de a scrie corpul.

Enunțul spune: „*subprogramul furnizează prin parametrul `k` o valoare*”. „Furnizează prin parametru” = parametru de ieșire = trebuie să fie referință, `int &k`. Așa cum l-ai scris, `int k`, funcția scrie într-o copie care dispare la `}` — exact B1 din runda 1, pe care l-ai închis corect în `varianta24.h` acum patru zile. Aici l-ai rescris.

Și dacă `k` e ieșirea, funcția nu mai are ce să returneze: antetul trebuie să fie `void`, nu `int`. Un `int` care nu returnează nimic e comportament nedefinit — B7 din runda 1, tot închis, tot rescris aici.

`p = 1` de pe linia 18 arată că știai unde mergi: `p` e puterea lui 10 la care adaugi următoarea cifră pară, și se înmulțește cu 10 **doar când chiar iei o cifră**. Ce lipsește e corpul lui `if` și cazul „nicio cifră pară”.

```
// ACUM
int f(int a[], int n, int k) {
    int p = 1;
    k = 0;
    for (int i = n-1; i >= 0; i--) {
        if (a[i] % 2 == 0) {
            }
        }
    }

// CORECT
void f(int a[], int n, int &k) {
    int p = 1;
    k = 0;
    bool gasit = false;
    for (int i = 0; i < n; i++) {          // a[0] = unitatile
        if (a[i] % 2 == 0) {
            k = k + a[i] * p;
            p = p * 10;
            gasit = true;
        }
    }
    if (!gasit) k = -1;
}
```

Atenție la sensul parcurgerii: enunțul spune că `a[0]` e cifra unităților. Deci mergi de la `0` în sus și înmulțești `p` pe măsură — nu de la `n-1` în jos.

B6 — `varianta26.h`: afișează un `0` în plus la final

`varianta26.h:38-51` (`solutie4b`)

CUM E ACUM: pe exemplul din enunț:

10 2 19 9 15 0 trebuia 10 2 19 9 15

DE CE: `sterge` e **corect**. L-am rulat izolat, pe convenția din enunț (elemente numerotate de la 1):

```
v[1..5] = 5 7 7 9 11 -> sterge(v, 5, 2, 3) -> n=3, 5 9 11
```

Bug-ul e în `solutie4b`, și e unul singur cu trei fețe: în 14 linii folosești trei convenții de numerotare.

1. **Datele.** Fișierul `NUMERE.IN` are pe prima linie `12` — acela e `n`, nu un element. Tu ai pus toate cele 13 numere în tablou și ai scris `n = 13`. Deci `12` a devenit primul element al șirului.
2. **Căutarea dublurilor** merge de la `0`: `for (int i = 0; i < n; i++)` compară `a[i]` cu `a[i+1]`, adică numără de la zero.
3. **Afișarea** merge de la `1`: `for (int i = 1; i <= n; i++)`.

Afișarea de la `1` sare peste `a[0]` — adică peste `12` -ul pus din greșeală — și de-aia rezultatul *pare* corect. Dar merge cu unul mai departe la capăt și tipărește `a[n]`, care e un zero din tabloul neinițializat. Cele două greșeli se anulează la început și se adună la sfârșit.

Ai două soluții curate. Ori totul de la `0` (și atunci `sterge` trebuie scris tot de la `0`), ori totul de la `1` — a doua e mai aproape de enunț, fiindcă enunțul însuși numerotează de la 1.

```
// ACUM
int n = 13;
int a[101] = { 12, 10, 10, 2, 2, 19, 9, 9, 9, 9, 15, 15, 15 };
for (int i = 0; i < n; i++) { ... }
for (int i = 1; i <= n; i++) cout << a[i] << " ";

// CORECT — totul de la 1, ca in enunt
int n = 12;
int a[101] = { 0, 10, 10, 2, 2, 19, 9, 9, 9, 9, 15, 15, 15 };
for (int i = 1; i < n; i++) {
    if (a[i] == a[i+1]) { sterge(a, n, i+1, i+1); i--; }
}
for (int i = 1; i <= n; i++) cout << a[i] << " ";
```

B7 — `varianta52.h`: k iese cu 1 mai mic când tot șirul e multime

`varianta52.h:37-44`

CUM E ACUM: pe datele tale merge, pe altele nu:

```
{16,17,8,31,8,2,10} -> k = 4      corect (8 se repeta pe pozitia 5)
{1,2,3}               -> k = 2      trebuia 3
```

DE CE: `mult(i, a)` verifică **primele** `i` elemente — deci valoarea testată e chiar `i`. Bucla ta merge `for (int i = 0; i < d; i++)`, adică `i` ia valorile `0, 1, ..., d-1`. Valoarea `d` nu e testată niciodată. Dacă tot șirul e mulțime, răspunsul corect e `d`, iar tu nu ajungi să-l încerci.

Pe datele tale nu se vede, fiindcă acolo răspunsul e `4`, mult sub `7`, iar `break`-ul oprește bucla înainte de capăt. Bug-ul apare numai când niciun `break` nu se declanșează — adică exact pe cazul limită.

E același lucru pe care l-ai făcut bine în runda 8 pe `vector-vectori`: datele de test trebuie alese ca să lovească marginea, nu ca să confirme mijlocul.

```
// ACUM
for (int i = 0; i < d; i++) {

// CORECT
for (int i = 0; i <= d; i++) {
```

🟡 Importante

M1 — 9 variante nu citesc datele de unde cere enunțul

`varianta3.h` · `varianta23.h:38` · `varianta24.h:56` · `varianta26.h:39` · `varianta39.h:21` ·
`varianta40.h:18` · `varianta46.h:17` · `varianta51.h:21` · `varianta52.h:33`

În tot repo-ul nu există niciun `#include <fstream>`. Șase enunțuri cer explicit fișier — `bac.in` (v39, v40), `bac.txt` (v46), `NUMERE.IN` (v26), `date.in` (v51, v52) — și trei cer tastatura (v3, v23, v24).

Distincția care contează: la un enunț care spune doar „*Scriveți definiția completă a subprogramului X*” (v18, v19, v41, v42, v43, v64, v66, v52a), un driver cu date scrise în cod e perfect în regulă — subprogramul e livrabilul, restul e bancul tău de test. La punctul b), programul **e** livrabilul, iar sursa datelor face parte din el.

Citirea din fișier are exact trei linii în plus față de `cin`:

```
#include <fstream>
ifstream fin("bac.in");
fin >> n;
for (int i = 0; i < n; i++) fin >> v[i];
fin.close();
```

M4 din runda 1 (`varianta39`) intră aici.

M2 — coliziunea de nume s-a agravat

M1 din runda 1, măsurat acum:

<code>solutie3</code>	17 fișiere	<code>p</code>	3 fișiere
<code>solutie4</code>	6 fișiere	<code>f</code>	2 fișiere
<code>solutie3b</code>	2 fișiere	<code>sort</code>	2 fișiere
<code>solutie4b</code>	2 fișiere	<code>sub</code>	2 fișiere

Cu 30 de variante, două incluse simultan înseamnă `error: redefinition`. Azi merge pentru că `varianteBac.cpp` include un singur header — dar asta înseamnă și că nu poți rula două variante fără să editezi `varianteBac.cpp` de fiecare dată.

Cel mai ieftin fix, pe care îl aplici o dată și scapi: pune fiecare header într-un `namespace`, iar în `varianteBac.cpp` apelezi cu prefix.

```
// varianta42.h
namespace v42 {
    int p(int a[], int n, int k) { ... }
    void solutie3() { ... }
}

// varianteBac.cpp
#include "varianta42.h"
int main() { v42::solutie3(); }
```

M3 — `varianta42.h`: `p` distruge tabloul apelantului

`varianta42.h:18–31`

Subprogramul trebuie să *returneze* o sumă. El sortează `a` descrescător pe loc, iar `a` e un pointer la tabloul din `solutie3`:

```
inainte:  5 2 5 4 1 3
p(a,6,4) = 17          corect
dupa    :  5 5 4 3 2 1
```

Cine apelează `p` și vrea după aceea să afișeze șirul citit, află că nu-l mai are. Copiază într-un tablou local și sortează copia.

M4 — `varianta46.h`: `k` nu se citește, tabloul nu se construiește

`varianta46.h:17–19`

Enunțul: „citește de la tastatură un număr natural k ... **construiește în memorie** și afișează pe ecran un tablou *unidimensional*”. Tu afișezi pe măsură ce parcurgi, fără să construiești nimic, iar `k = 5` e scris în cod. Rezultatul e corect (`100 400 56`), cerința nu e acoperită.

M5 — `varianta55.h`: `ordin(0)` dă 1, concatenarea cu 0 pierde cifra

`varianta55.h:20-27`

```
n=1 / a = 5 / b = 0    -> 5          trebuia 50
```

`while (nr != 0)` nu intră deloc pentru `nr = 0`, deci `p` rămâne `1` și `5 * 1 + 0 = 5`. Zero are o cifră, deci `ordin(0)` trebuie să fie `10`. O gardă la început: `if (nr == 0) return 10;`.

Restul e corect și e o rezolvare bună — `712 534 9100` pe trei perechi.

M6 — prompturile de la tastatură (M3 runda 1, neatins)

`varianta1.h:68,71` · `varianta6.h:20` · `varianta11.h:21` · `varianta12.h:54,57`

Aceleași cinci locuri ca acum o săptămână. `n=` în fața răspunsului înseamnă test picat la comparația automată.

M7 — `varianta20.h`: nule încă afișează (M5 runda 1, neatins)

`varianta20.h:29-31` — subprogramul doar rearanjează; afișarea e treaba programului principal.

Cleanups

- **C1** — `varianta23.h:31` : bucla merge `i < n` și citește `x[n]`, în afara zonei permutate. Valoarea se pierde imediat (o suprascrie `x[n-1] = aux`), deci azi nu se vede — dar la `n = 100` pe un tablou `x[100]` citești în afara tabloului. Corect: `i < n - 1`.
 - **C2** — `varianta18.h:18` : `count` intră în concurență cu `std::count`, adus de `using namespace std`. Azi trece fiindcă `<algorithm>` nu e inclus — exact povestea lui `sort` de la M2 runda 1. Un nume propriu (`nrPesteMedie`) încheie subiectul.
 - **C3** — variabile declarate și nefolosite: `varianta24.h:59` (`ct`), `varianta3.h:55,81,82` (`nou`, `n`, `f`), `varianta25.h:18` (`p`), `varianta8.h:21,23,24` (`j`, `paritate`, `i`). Fiecare e un semn că acolo era de scris ceva.
 - **C4** — `varianta5.h:27` : `int f(long a)` — pe ramura `return a` valoarea se îngustează tăcut. `long f(long a)`. (C3 runda 1)
 - **C5** — `<cmath>` e inclus în **toate cele 30** de fișiere și nu e apelat nimic din el nicăieri. `Header.h` conține doar `#pragma once` și nu e inclus de nimeni. (C4 runda 1)
 - **C6** — `///?` rămase în `varianta3.h:6`, `varianta5.h:6`, `varianta8.h:6`, `varianta20.h:6`, `varianta24.h:6`. (C5 runda 1) — a doua oară când întreb: ce înseamnă? Dacă e „aici nu sunt sigur”, spune-mi care sunt și le luăm la lecție.
 - **C7** — spațiu în plus la finalul liniei: `cout << v[i] << " "` lasă un spațiu după ultimul element, în `v5, v19, v23, v26, v46, v51, v55, v62, v8`. Unele evaluatoare îl iartă, altele nu. Varianta sigură: `if (i > 0) cout << " "; cout << v[i];`
 - **C8** — `varianta24.h:49` : `stergere` merge `i < dim` și citește `x[dim]`, același tipar ca C1.
 - **C9** — neatinse din runda 1: `varianta16.h:19` (`a[i] % k` cu `k = 0`), `varianta14.h:22` (`fr[cif]++` fără gardă pe interval).
-

✓ Ce e corect — verificat prin rulare

fișier	test	rezultat
varianta18.h	{12, 7.5, 6.5, 3, 8.5, 7.5}	4 ✓
varianta19.h	{12, -7.5, 6.5, -3, -8, 7.5}	-7.5 -3 -8 12 6.5 7.5 ✓
varianta23.h	{1,2,3,4,5}	5 4 3 2 1 ✓
varianta39.h	ambele cazuri	DA / NU ✓
varianta40.h	{10,1133,12,331,42,1354,221,13}	DA ✓
varianta41.h	{23,44,76,26,65}	1 ✓
varianta42.h	{5,2,5,4,1,3} , k=4	17 ✓
varianta43.h	{3,12,7,1,4,3}	4 ✓
varianta46.h	{100,9,400,56,7,10} , k=5	100 400 56 ✓
varianta51.h	{16,22,3,6,1}	4 perechi ✓
varianta52.h	{16,17,8,31,8,2,10}	4 ✓
varianta55.h	12 5 100 / 7 34 9	712 534 9100 ✓
varianta62.h	n = 8	1 1 3 5 13 21 55 89 ✓
varianta64.h	{4,8,3,9,4}	4 ✓
varianta66.h	{12,36,48,6,60}	6 ✓
varianta5.h	1125 2 314 101 37 225 12	2 37 101 ✓

Patru lucruri merită spuse pe nume, pentru că nu sunt rezolvări de manual:

varianta23.h — inversarea prin apeluri repetate de `shift`. Asta e exact ce voia enunțul și e partea grea a problemei. Ai văzut că o permutare circulară pe primele `i` elemente, aplicată pentru `i` de la `n` în jos până la `2`, inversează tabloul. Nu e evident și nu se ghicește — se desenează. Ai făcut-o.

varianta66.h — Euclid pe tot tabloul. `cmmdc(a, b, c) = cmmdc(cmmdc(a, b), c)` aplicat în lanț, cu algoritmul prin resturi scris corect, fără recursivitate și fără scăderi repetate. Curat.

varianta55.h — `ordin()`. Ai izolat „de câte ori înmulțesc cu 10 ca să fac loc” într-un subprogram propriu, în loc să numeri cifrele în mijlocul formulei. E fix decizia pe care un profesor o caută la subiectul III.

varianta62.h — `fibo`. Ai înțeles că `v` se umple doar cu termenii impari, dar recurența trebuie să meargă peste **toți** termenii. De-aia `ct` avansează numai în `if`, iar `a` și `b` avansează întotdeauna. Două contoare cu ritmuri diferite în aceeași buclă — e genul de lucru la care se greșește.

Q&A — răspunde-mi la astea trei

1. `varianta8.h`, `interclasare`: pe `a={2,4,8,10,14}` și `b={3,5,11}` iese corect, pe `a={2,4}` și `b={1}` iese `1` în loc de `1 2`. **Fără să rulezi nimic**: ce valoare are `j` în momentul în care se evaluează prima dată condiția `while (i < dimA && j < dimB)`, în fiecare dintre cele două cazuri? Și de ce e diferită de ce înseamnă `j` în interiorul buclei?
2. `varianta25.h`: ai scris `int f(int a[], int n, int k)`. Enunțul spune „subprogramul furnizează **prin parametrul k** o valoare”. **De ce antetul ăsta nu poate face ce cere enunțul**, oricât de bine ai scrie corpul? Ai închis exact bug-ul ăsta în `varianta24.h` acum patru zile — ce e diferit aici?
3. `varianta52.h` merge pe `{16,17,8,31,8,2,10}` și greșește pe `{1,2,3}`. **Ce au datele tale și nu au ale mele**, de ascunde bug-ul? Și, mai important: cum arată, în general, setul de date care scoate la iveală o greșeală de tip „buclo se oprește cu unul prea devreme”?

Ce ai de făcut, în ordine

Întâi cele care se rezolvă într-o linie — sunt patru și închid patru constatări:

1. `varianta5.h:54` — `NU EXISTA` (B4)
2. `varianta52.h:37` — `i <= d` (B7)
3. `varianta23.h:31` — `i < n - 1` (C1)
4. `varianta55.h:21` — gardă `if (nr == 0) return 10;` (M5)

Apoi cele de rescris:

5. `varianta26.h` — B6: alege o singură convenție de numerotare și du-o până la capăt în tot fișierul.
6. `varianta24.h` — B3: scoate `mini / maxi` din afișare, pune `fixed << setprecision(3)`, citește `n` și `x` de la tastatură.
7. `varianta8.h` — B2: fă cele două convenții una singură, apoi mută `interclasare` în `sub`, cu antetul din enunț.
8. `varianta25.h` — B5: `void f(int a[], int n, int &k)` și corpul lui `if`.
9. `varianta3.h` — B1: termină ce ai început, dar cu `cif` apelat.

La final, o dată pentru tot repo-ul:

10. M2 — `namespace` în fiecare header. E mecanic, dar după el poți rula orice variantă fără să editezi `varianteBac.cpp`.
 11. M1 — citirea din fișier pe cele 6 variante care o cer. Începe cu `varianta39.h`, e cea mai simplă.
- M6, M7 și C4–C9 le lăsăm pentru runda 4 — nu te blochează.

Runda 1 — 2026-09-15 — sha 46a53db

Commit 46a53db tema (variantele 3, 5, 24, 39) plus verificarea integrală a celor din 4281854 1st push . Poarta de compilare: trecută, 14/14 headere, 4 warning-uri din care 3 bug-uri reale.

Verdict: din cele 4 fișiere ale temei, unul singur (varianta39) producea răspunsul cerut. Tiparul rundeii: *te oprești la un pas de final* — algoritmul scris, linia care îl leagă de răspuns lipsă.

#	constatare	fișier	stare la runda 2
B1	parametrii de ieșire transmiși prin copie, nu prin referință	varianta24.h:18	✅ închis
B2	<code>for (int i = i; ...)</code> — bucla pornește de la o valoare nedefinită	varianta24.h:23	✅ închis
B3	nu afișează nimic; media s-ar calcula în întregi	varianta24.h:57	🟡 pe jumătate → B3 runda 2
B4	cifra 9 nu e numărată niciodată (<code>i < 9</code> pe un tablou de 10)	varianta3.h:48,62	⬜ dispărut odată cu rescrierea
B5	palindrom: <code>0</code> nu se afișează, zeroul lipsește, mijlocul lipsește	varianta3.h:50–66	🔴 înrăutățit → B1 runda 2
B6	punctul b) nu e scris: <code>f</code> nu e apelat niciodată; mesaj greșit	varianta5.h:44	🟡 pe jumătate → B4 runda 2
B7	funcție <code>int</code> care nu întoarce nimic	varianta39.h:15	✅ închis
B8	<code>sub</code> e schelet: interclasează și nu afișează nimic	varianta8.h:18–33	🔴 neatins → B2 runda 2
M1	două variante nu pot exista în același proiect (nume duplicate)	tot repo-ul	🔴 agravat → M2 runda 2

#	constatare	fișier	stare la runda 2
M2	<code>sort</code> definit de două ori și suprapus peste <code>std::sort</code>	<code>varianta1.h:44</code> , <code>varianta12.h:28</code>	● neatins
M3	prompturile de la tastatură strică evaluarea automată	5 locuri	● neatins → M6 runda 2
M4	<code>varianta39</code> cere citire din <code>bac.in</code> , nu din <code>cin</code>	<code>varianta39.h:21</code>	● neatins → M1 runda 2
M5	subprogramul <code>nule</code> afișează, deși n-ar trebui	<code>varianta20.h:29-31</code>	● neatins → M7 runda 2
C1	<code>a[i] % k</code> cu <code>k = 0</code>	<code>varianta16.h:19</code>	● neatins
C2	<code>fr[cif]++</code> fără gardă pe interval	<code>varianta14.h:22</code>	● neatins
C3	<code>int f(long a) —</code> îngustare tăcută pe <code>return a</code>	<code>varianta5.h:32</code>	● neatins → C4 runda 2
C4	<code><cmath></code> nefolosit peste tot; <code>Header.h</code> orfan	tot repo-ul	● neatins → C5 runda 2
C5	marcaje <code>///?</code> rămase	5 fișiere	● neatins → C6 runda 2

Corecte la runda 1, verificate prin rulare: `varianta1.h` (b), `varianta7.h` , `varianta12.h` , `varianta16.h` , `varianta17.h` , `varianta20.h` , `varianta39.h` , plus `varianta6.h` , `varianta11.h` , `varianta14.h` fără obiecții de fond.



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro